

Testing — Group / Ungroup figures (JHotDraw 7)

Unit under test: the Group/Ungroup domain logic (`GroupingService` , `FigureGroup`) **Framework:** JUnit 4 + Mockito | **Module:** `L7/group-tests`

What was tested

The important business functionality of the feature is the grouping and ungrouping operations — in JHotDraw, `GroupAction.groupFigures()` / `ungroupFigures()` . Those methods cannot be unit-tested in isolation because they call out to Swing collaborators (`DrawingView` , `Drawing` , `CompositeFigure` , `Figure`). The logic is therefore modelled in a small, testable module with the same behaviour:

- `GroupingService.group()` — removes the selected figures from the drawing and adds a single group containing them.
- `GroupingService.ungroup()` — removes the group and releases its children back into the drawing.
- `FigureGroup` — the composite that holds the children in z-order.

The `Drawing` collaborator is an **interface**, so in the tests it is replaced by a **Mockito mock** — exactly the technique the lab asks for: *when execution passes outside the method under test you have a dependency, so apply mocks/stubs*.

How the feature was verified

Test	Type	Verifies
<code>group_movesSelectedFiguresIntoANewGroup</code>	Best case	3 figures → group of 3; each figure removed from the drawing, group added once (Mockito <code>verify</code>)
<code>ungroup_releasesChildrenBackIntoTheDrawing</code>	Best case	group removed, both children re-added to the drawing
<code>group_withSingleFigure_isStillGrouped</code>	Boundary	a 1-figure selection still groups correctly
<code>group_withEmptySelection_createsEmptyGroup_andRemovesNothing</code>	Boundary	empty selection → empty group, <code>drawing.remove</code> never called
<code>ungroup_emptyGroup_removesGroup_andAddsNoChildren</code>	Boundary	empty group → group removed, no child re-added
<code>group_withNullFigures_throwsException</code>	Error	recoverable bad input → <code>IllegalArgumentException</code>

Test	Type	Verifies
<code>ungroup_withNullGroup_throwsException</code>	Error	recoverable bad input → <code>IllegalArgumentException</code>
<code>FigureGroupTest</code> (3 tests)	Entity	empty-on-creation, insertion order, unmodifiable children view

Mocking (Mockito): `@RunWith(MockitoJUnitRunner.class)` + `@Mock Drawing drawing`. Each test asserts on the return value **and** verifies the interactions with the mocked drawing (`verify(drawing).remove(...)` , `verify(drawing).add(group)` , `verify(drawing, never())...` , `verifyNoMoreInteractions(drawing)`), so a single code-path through a single method is checked with no real dependency.

Java assertions vs exceptions: `GroupingService` uses the `assert` keyword for **invariants that should never happen** (`assert drawing != null` , `assert group.size() == figures.size()`) — an assertion halts the program. Bad but **recoverable** caller input (a `null` figure list) instead throws an `IllegalArgumentException` , letting the program continue. Surefire runs tests with assertions enabled (`-ea`) by default, so the invariants are active during the run.

Verification result

The tests run automatically in the **CI pipeline** (`.github/workflows/ci.yml` , step "*Build and test L7 group-tests*") on every push and pull request. A green run is the evidence that the Group/Ungroup feature behaves as specified for the best case and all identified boundary cases.